

Experiment No.9

Aim: To study and implement Database as a Service on AQL/NOSQL databases like AWS RDS, AZURE SQL/MongoDB Lab/Firebase.

Objectives:

- Develop a theoretical understanding of DBaaS principles, focusing on the benefits of managed database services and the differences between SQL and NoSQL databases.
- Apply practical knowledge to configure and utilize AWS RDS for SQL databases and DynamoDB for NOSQL databases.

Outcomes:

- To demonstrate proficiency in utilizing AWS RDS and DynamoDB to create, manage, and optimize SQL and NoSQL databases.
- Apply knowledge in implementing Database as a Service, understanding how to choose between SQL and NOSQL databases based on use cases and requirements.

Software Requirements: Windows/Ubuntu OS, AWS Account.

Theory:

Database as a Service (DBaaS) provides managed database solutions in a cloud environment, relieving users from the complexities of database administration. Amazon RDS is a managed relational database service that supports SQL databases such as MySQL, PostgreSQL, and SQL Server. Amazon DynamoDB is a fully managed NoSQL database service.

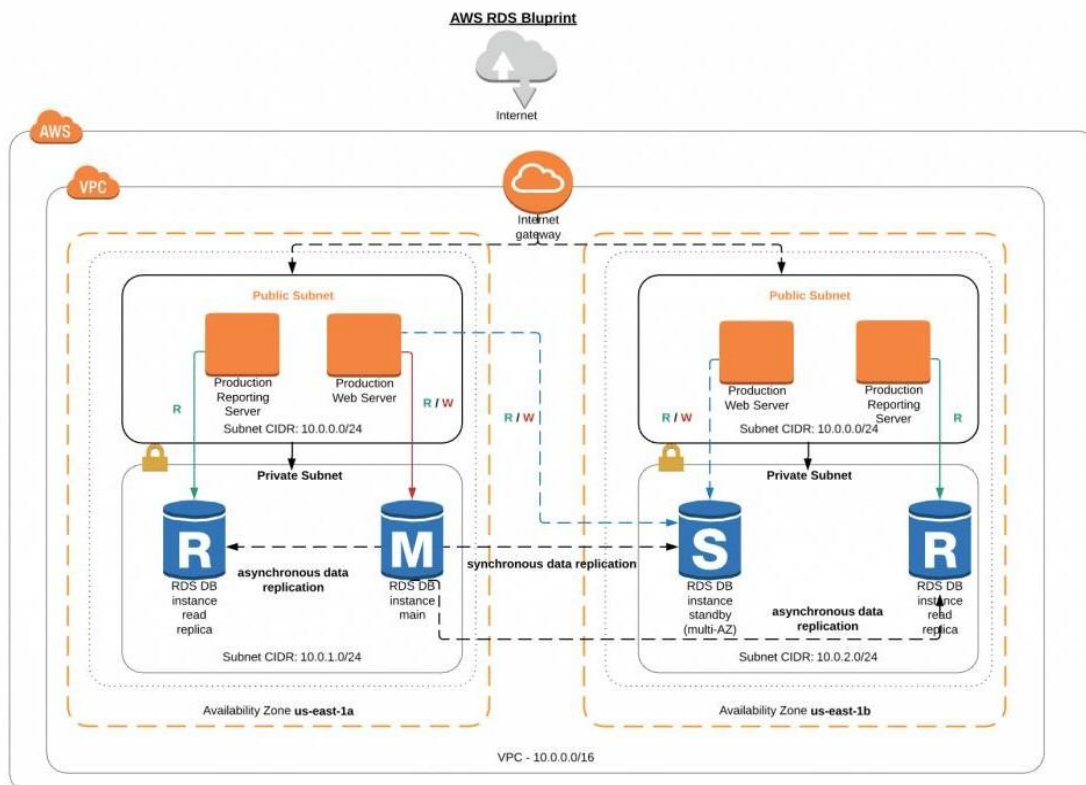


Figure 9 Amazon RDS architecture

Key Components:

1. **AWS Cloud Infrastructure:** The foundation of RDS, providing compute, storage, and networking resources.
2. **Database Instances:** Managed database environments running on virtual machines within the AWS cloud.
3. **Database Engines:** RDS supports six different database engines: MySQL, PostgreSQL, MariaDB, Oracle, Microsoft SQL Server, and Amazon Aurora.
4. **Availability Zones:** RDS instances can be deployed across multiple availability zones for high availability and fault tolerance.
5. **Security Groups:** Virtual firewalls control access to database instances, ensuring only authorized users and applications can connect.
6. **Backup and Recovery:** RDS automatically backs up databases and provides point-in-time recovery options for data protection.
7. **Monitoring and Logging:** RDS offers tools to monitor database performance, usage, and logs for troubleshooting and optimization.
8. **Read Replicas:** RDS allows creating read-only replicas of database instances to distribute read traffic and improve performance.
9. **Parameter Groups:** Collections of configuration settings that control database behavior, allowing customization.
10. **Option Groups:** Provide additional features and capabilities, such as performance insights and enhanced security.

How RDS Works:

1. **Instance Creation:** You select a database engine and specify desired instance configuration (instance class, storage, etc.).
2. **Infrastructure Provisioning:** AWS provisions the underlying infrastructure and database software.
3. **Database Endpoint:** A unique endpoint is assigned to the instance for database connection.
4. **Data Access:** You connect to the database instance using standard database tools and APIs.
5. **Database Management:** Perform CRUD operations on data within the database.
6. **Maintenance and Monitoring:** AWS handles infrastructure maintenance, backups, security patching, and instance monitoring.

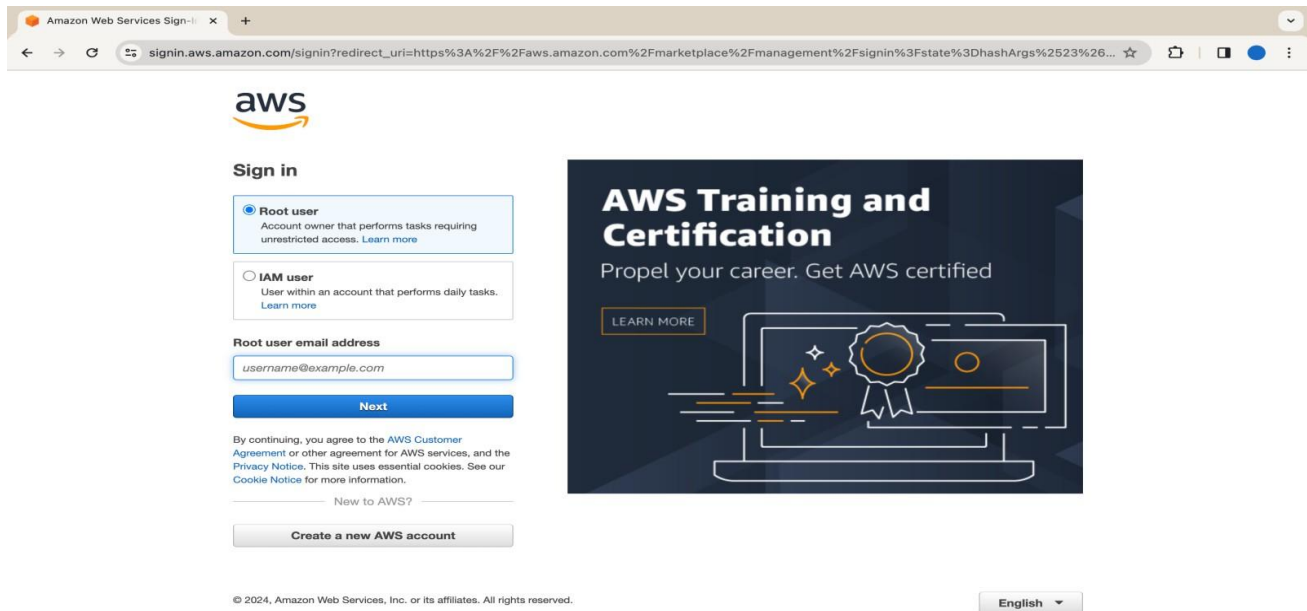
Benefits of RDS:

- **Managed Service:** AWS manages database infrastructure, reducing operational overhead.
- **Scalability:** Easily scale database resources up or down as needed.
- **High Availability:** Multi-AZ deployments ensure database availability even in the event of outages.
- **Security:** RDS implements robust security measures to protect databases.
- **Cost-Effectiveness:** Pay only for the database resources you use.
- **Multiple Database Engines:** Choose from a variety of popular database engines.
- **Ease of Use:** Simple to set up and manage, with a user-friendly console and APIs.

Stepwise Procedure to Implement Database as a Service on AWS RDS and DynamoDB:

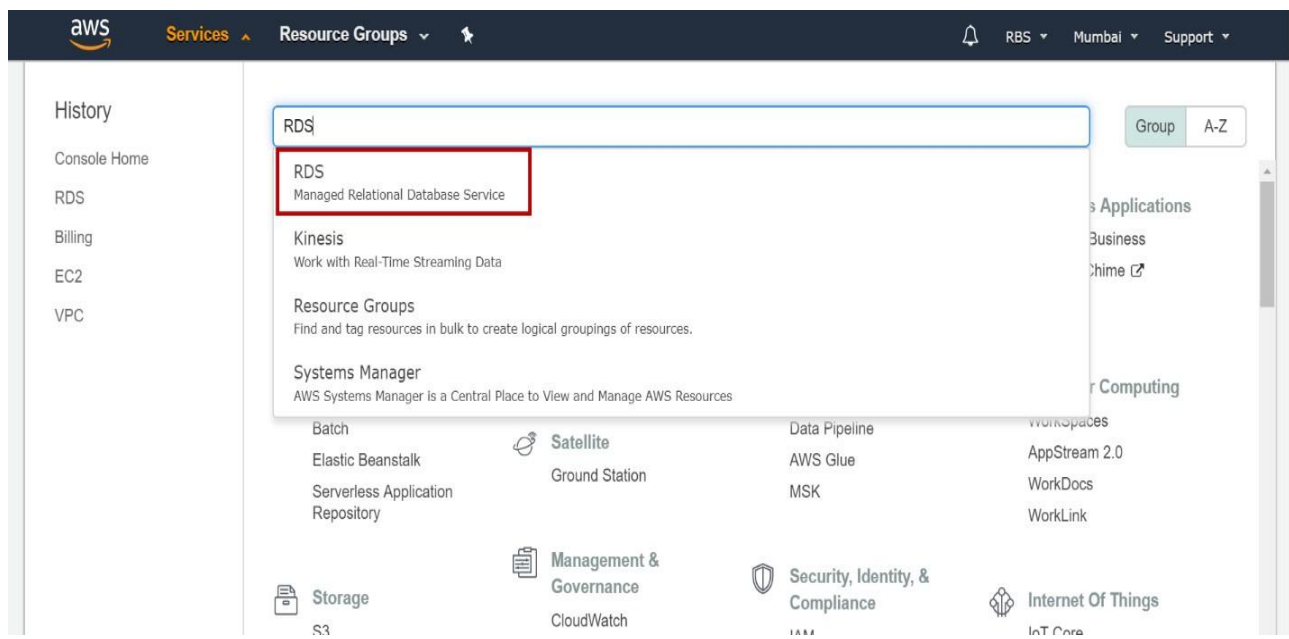
Step 1: Sign into AWS Console

- Log in to the AWS Management Console using your AWS account credentials.



Step 2: Access RDS Service

- Navigate to the RDS service in the AWS Console.



Step 3: Create RDS Instance (SQL Database)

- Create an RDS instance for a SQL database (e.g., MySQL).
- Configure instance settings, including database engine, version, and credentials.

Create database

Choose a database creation method [Info](#)

☒ **Standard Create**
You set all of the configuration options, including ones for availability, security, backups, and maintenance.

☐ **Easy Create**
Use recommended best-practice configurations. Some configuration options can be changed after the database is created.

Engine options

Engine type [Info](#)

☐ Amazon Aurora


☒ **MySQL**


☐ MariaDB


☐ PostgreSQL


☐ Oracle


☐ Microsoft SQL Server


Edition

☒ **MySQL Community**

Step 4: Access DynamoDB Service

- Navigate to the DynamoDB service in the AWS Console.



Amazon DynamoDB

Amazon DynamoDB is a fast and flexible NoSQL database service for all applications that need consistent, single-digit millisecond latency at any scale. Its flexible data model and reliable performance make it a great fit for mobile, web, gaming, ad-tech, IoT, and many other applications.

[Create table](#)

[Getting started guide](#)



Create tables

Create DynamoDB tables with a few clicks. Just specify the desired read and write throughput for your table, and DynamoDB handles the rest.

[More about DynamoDB throughput](#)



Add and query items

Once you have created a DynamoDB table, use the AWS SDKs to write, read, modify, and query items in DynamoDB.

[DynamoDB API reference](#)



Monitor and manage tables

Using the AWS Management Console, you can monitor performance and adjust the throughput of your tables, enabling you to scale seamlessly.

[Monitoring tables](#)

DynamoDB documentation & support

[Getting started guide](#) | [FAQ](#) | [Release notes](#) | [Developer guide](#) | [Forums](#) | [Report an issue](#)

Step 5: Create DynamoDB Table (NoSQL Database)

- Create a DynamoDB table for a NoSQL database.
- Configure table settings, including primary key attributes and capacity settings.

The screenshot shows the 'Create DynamoDB table' wizard in the AWS Management Console. The table name is 'simplilearn_users'. The primary key is 'LastName' with a 'String' data type. The 'Table settings' section shows 'Use default settings' selected. The 'Secondary indexes' section has a '+ Add index' button. The 'Provisioned capacity' section shows 'Read capacity units' set to 1 and 'Write capacity units' set to 1000. The estimated cost is \$483.70 / month. At the bottom, there are 'Cancel' and 'Create' buttons.

Create DynamoDB table

DynamoDB is a schema-less database that only requires a table name and primary key. The table's primary key is made up of one or two attributes that uniquely identify items, partition the data, and sort data within each partition.

Table name* simplilearn_users ⓘ

Primary key* Partition key

LastName String ⓘ

☐ Add sort key

Table settings

Default settings provide the fastest way to get started with your table. You can modify these default settings now or after your table has been created.

☒ Use default settings

Secondary indexes

Name	Type	Partition key	Sort key	Projected Attributes
------	------	---------------	----------	----------------------

+ Add index

Provisioned capacity

Read capacity units 1 Table

Write capacity units 1000 Table

Estimated cost \$483.70 / month (Capacity calculator)

Additional charges may apply if you exceed the AWS Free Tier levels for CloudWatch or Simple Notification Service. Advanced alarm settings are available in the CloudWatch management console.

Cancel Create

Step 6: Insert Data into SQL and NoSQL Databases

- Insert sample data into the SQL database hosted on RDS.
- Insert sample data into the NoSQL table hosted on DynamoDB.

The screenshot shows the 'Create item' dialog for the 'simplilearn_users' table. The dialog lists the following attributes: 'LastName String : Smith', 'FirstName String : Wendy', and 'Position String : Manager'. At the bottom, there are 'Cancel' and 'Save' buttons.

Create item

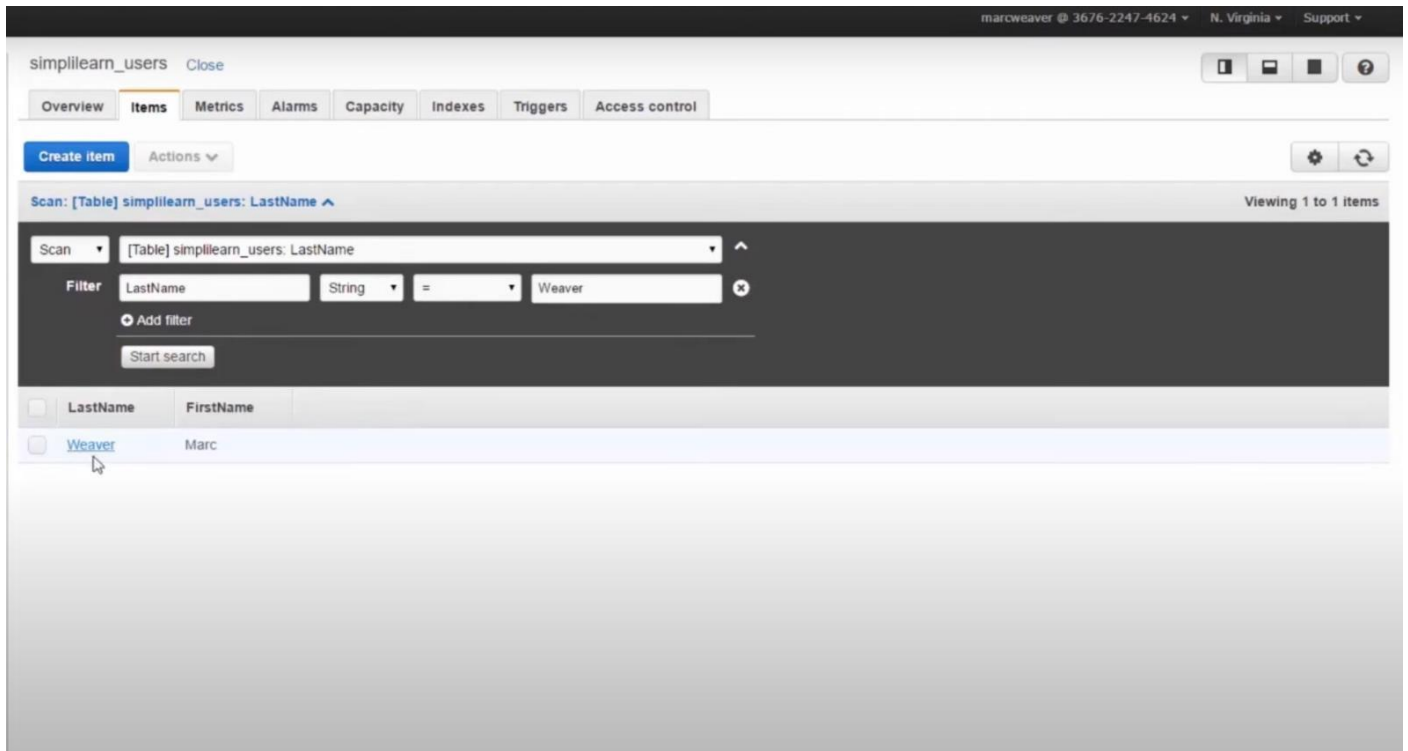
Tree -

- Item [1]
 - LastName String : Smith
 - FirstName String : Wendy
 - Position String : Manager

Cancel Save

Step 7: Query and Retrieve Data

- Execute SQL queries to retrieve data from the SQL database.
- Use the AWS SDK or Console to query and retrieve data from the DynamoDB table.



Conclusion:

Thus, we have understood and implemented Database as a Service using AWS RDS for SQL database and DynamoDB for NoSQL databases.